

DW_dpll_sd

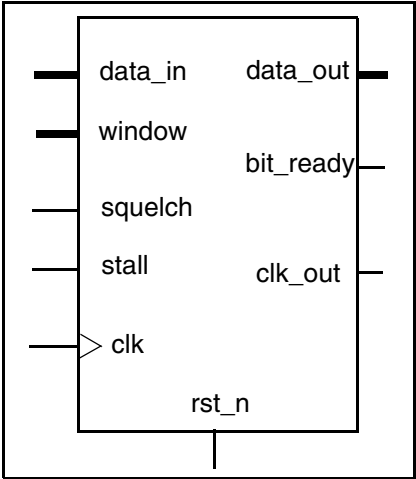
Digital Phase Locked Loop

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

Features and Benefits

- Parameterizable divisor (ratio of reference clock to baud rate)
- Multichannel data recovery (recovery of channels that accompany the locked channel)
- Stall input for power saving mode and/or prescaler (allowing one DW_dpll_sd to recover data at multiple rates)
- Squelch input for ignoring phase information when channel data is unknown or unconnected
- Sampling window control to aid data recovery under harsh conditions
- Parameterizable gain to meet a variety of application needs
- Parameterizable filter (controls phase correction reactivity from minor phase errors)

Revision History



Applications

- Networking
- Digital communication

Description

DW_dpll_sd is a digital phase-locked loop (DPLL) designed for data recovery.

Table 1-1 Pin Description

Pin Name	Width	Direction	Function
clk	1 bit	Input	Reference clock
rst_n	1 bit	Input	Asynchronous reset, active low
stall	1 bit	Input	Stalls everything except synchronizer, active high
squelch	1 bit	Input	Turns off phase detection. When high no phase correction is carried out leaving DPLL free running, active high

Table 1-1 Pin Description (Continued)

Pin Name	Width	Direction	Function
window	$\text{ceil}(\log_2(\text{windows}))$	Input	Sampling window selector ^a
data_in	<i>width</i> bits	Input	Serial input data stream
clk_out	1 bit	Output	Recovered clock
bit_ready	1 bit	Output	Output data ready flag
data_out	<i>width</i> bits	Output	Recovered output data stream

a. The minimum value must be 1.

Table 1-2 Parameter Description

Parameter	Values	Description
width	1 to 16 Default: 1	Number of input serial channels
divisor	4 to 256 Default: 4	Determines the number of samples per input clock cycle
gain	1 to 2 Default: 1	Phase correction factor for the absolute value of clock phase error greater than 1 1: 50% phase correction 2: 100% phase correction
filter	0 to 8 Default: 2	Phase correction control for +/- 1 clock phase error region. 0: No correction 1: Always correct For integer $N > 1$, correct after N samples at a current phase (such as, N consecutive samples at +1 or N consecutive samples at -1)
windows	1 to $(\text{divisor}+1)/2$ Default: 1	Number of sampling windows for the input serial data stream

Table 1-3 Synthesis Implementations

Implementation Name	Function	License Feature Required
str	Synthesis model	DesignWare

The DPLL functionality is achieved through the use of a state machine design that runs on a reference clock that is at least four times the bit rate of the data channel. After synchronization of the `data_in` input signal to the reference clock (using a three stage synchronizer) the oversampled signal is monitored for transitions. When transitions are detected, the state machine evaluates (based on what state it's currently in) whether or

not an adjustment is needed to align the state machine's natural cycle to the incoming data stream. The amount of the adjustment depends on how far out of alignment the state machine is and the related parameter values used.

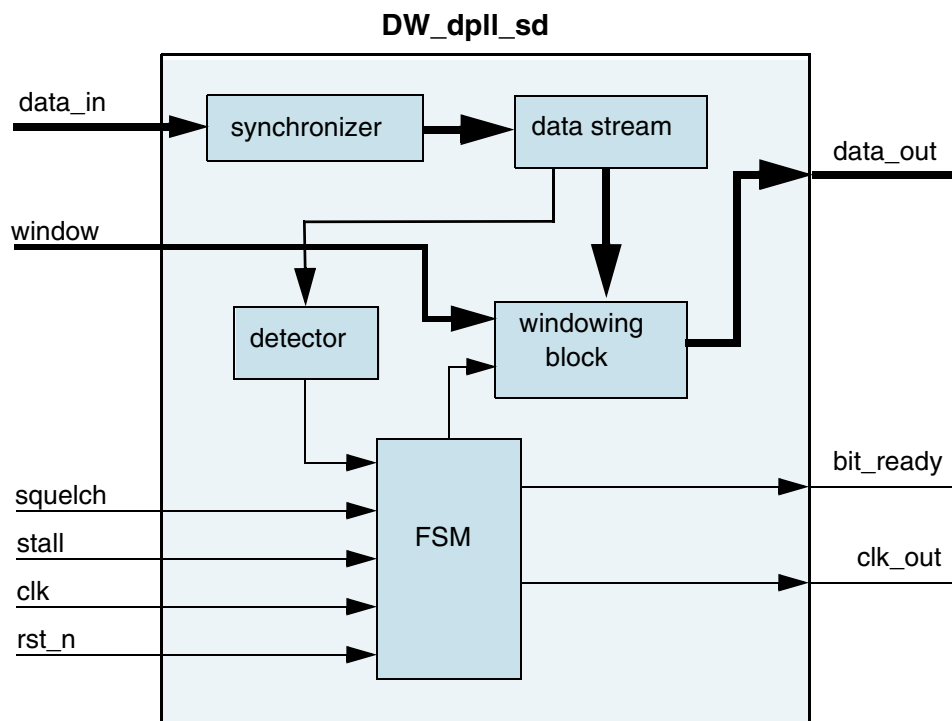
Recovered data is output on the `data_out` bus and the `bit_ready` output indicates the arrival of each data bit at the `data_out` bus. The `clk_out` port can provide a derived clock which is phase-aligned to cleanly clock recovered data from the `data_out` bus on its rising edge.

A `stall` input port is provided for power savings and for easily implementing multiple data rates through prescaling. A `squelch` input is provided to command the DPLL to ignore phase information during times when the link is not receiving valid data.

A windowing control bus is provided to allow control of where to sample data within a bit time. This may be useful in recovering noisy data especially if the data rate is significantly different from the DPLLs nominal rate.

The DW_dppll_sd can recover multiple channels of data as long as the channels are well phase-aligned with each other. See the *width* parameter functional description for more detail.

Figure 1-1 Functional Block



Functional Description

width -

The *width* parameter sets the number of serial input channels processed by the DW_dppll_sd component. Although as many as 16 channels can be recovered, phase locking operates only on the `data_in[0]` channel. Thus, all other channels received must be well phase-aligned with `data_in[0]` in order to cleanly recover data from them. Accompanying channels may be used to bring link status or other non-data information through the DPLL so as to keep this information aligned to the data stream.

divisor -

The static *divisor* parameter determines the number of divisions or phase states per bit time. This controls the sample rate, therefore, the granularity or coarseness of the phase corrections.

A *divisor* of 4, not only gives the coarsest granularity, but is the minimum number of samples per bit time to ensure an accurate sampling of the input signal. A large (*divisor*) value increases the number of correction (phase) states or divisions that a bit time is sub-divided into resulting in finer phase corrections.

Since each phase state is one reference clock period long, the number of phase states per bit time also determines the free running or center frequency of the DPLL. Increasing the *divisor* value increases the effective output clock period. For example, using a *divisor* value of 10 for a reference clock of 50 Mhz will divide the bit time into 10 states and set the center frequency to 5 Mhz.

The free running or nominal frequency should be chosen as close as possible to the input frequency or baud rate to ensure the best phase correction conditions.

stall -

The *stall* input port enhances the dynamics of the divisor to center frequency relationship. When using only the *divisor* parameter, compared with the addition of using the *stall* pin, each frequency is associated with only one *divisor* value to any one reference clock input. This means there is a set (inflexible) one-to-one correspondence between the *divisor* and center frequency.

Therefore, a large *divisor* value for a comparatively high reference clock to expected input frequency may be inconvenient especially if there is no desire to increase the granularity.

By using the *stall* pin to prescale the reference clock input, `clk`, a more flexible choice of sample rate can be chosen for a frequency.

Naturally, this allows for maintaining the same granularity over different frequencies or a coarser phase correction at a desired frequency balanced by a lower power system.

The resulting baud time will be the product of the *divisor* value and prescale factor. So, a *divisor* value of 5 for a reference clock of 50 Mhz, with the *stall* pin driven by a divide-by-two circuit translates to a center frequency of 5 Mhz.

Care should be taken in the use of pre-scaling to affect frequency and sampling rate combinations. Stalling the DPLL freezes all outputs at their current states. This could lead to output data mis-sampling if not considered. For example, stalling the DPLL right at the time the `bit_ready` output signal goes high will freeze it at that state (see [Figure 1-6](#) on page 11). So, for such a situation not gating the `bit_ready` signal with the *stall* signal, for instance, to generate a secondary data arrival flag will lead to incorrectly validating the `data_out` bus values if just reading the `bit_ready` output flag while in the stall state.

Input data passing through the synchronizer stage of the DPLL is unaffected by the *stall* pin.

filter -

The *filter* parameter controls the degree of adjustment in the plus or minus one phase error region, which represents the edge detection of input data transition one division off in either direction of the center or zero error reference for correction. This helps alleviate possible jitter propagation due to any dithering that occurs in this region.

gain -

The *gain* parameter determines the amount of phase correction for each detected phase error. A *gain* value of 1 results in a 50% phase correction of any detected error and a *gain* value of 2 in a 100% correction. A 50% phase correction (*gain* = 1) means a phase error of 6 will be corrected by 3 states compared with a 6 state correction for the case of a 100% phase correction (*gain* = 2).

windows -

The *windows* parameter feature allows for a number of sample points along the input stream to be used. A zero reference point at the approximate center of the bit time is chosen as the default sample point for output data. The reference point is the center of the bit time for even integer values of *divisor* and slightly off center for odd values. Additional sample points, if defined, are accessed through the *window* pin by use of indexes.

The index pattern for the window follows an alternating left and right sequence with the default zeroth index as the center pivot point, `window[0]`.

This windowing sequence is analogous to a spiral search pattern but in one dimension, where the goal is to maximize the probability of finding a good sample point assuming the center as the optimal sample point. For more information, see [Figure 1-3](#) on page 8.

This windowing option, provides the ability to compensate for noisy conditions such as asymmetric noise along the input stream and/or frequency difference or static error between the transmit and receive clock domains.

Application Notes

Note that with each phase correction that is carried out, the `clk_out` signal will either be shortened or lengthened by an amount equal to the phase correction.

Therefore it is generally desired to use the `bit_ready` output signal as a flag to indicate when `data_out` is ready to be read during normal operations.

Note that the `squelch` input disables the phase detection mechanism leaving the DPLL free running

Examples of Application Configurations

Three following common configurations can be used to cover most applications:

- For the non-coherent phase DPLL case and where there is more jitter with respect to the reference clock, *gain* = 1, *filter* = 2.
- For the coherent phase DPLL case, where the transmission and reception clock phase relation do not change with respect to each other, *gain* = 2, *filter* = 0.
- For the non-coherent phase DPLL condition and where there is less jitter and smoother corrections are desired, *gain* = 1, *filter* > 2.

Suppressing Warning Messages During Verilog Simulation

The Verilog simulation model includes macros that allow you to suppress warning messages during simulation.

To suppress all warning messages for all DWBB components, define the DW_SUPPRESS_WARN macro in either of the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_SUPPRESS_WARN
```

- Or, include a command line option to the simulator, such as:

+define+DW_SUPPRESS_WARN (which is used for the Synopsys VCS simulator)

The warning messages for this model include the following:

- If values other than 1 or 0 are present on a clock port, the following message is displayed:

```
WARNING: <instance_path>.<clock_name>_monitor:  
at time = <timestamp>, Detected unknown value, x, on <clock_name> input.
```

To suppress only this warning message for all DWBB components, use the following macro:

- Define the DW_DISABLE_CLK_MONITOR macro. You can define this macro in the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_DISABLE_CLK_MONITOR
```

- Or, include a command line option to the simulator, such as:

+define+DW_DISABLE_CLK_MONITOR (which is used for the Synopsys VCS simulator)

This message is also suppressed using the DW_SUPPRESS_WARN macro explained earlier.

Timing Diagrams

The following figures show various timing conditions for the DW_dpll_sd.

Figure 1-2 Functional Operation: Initialize Reset Followed by Phase Correction

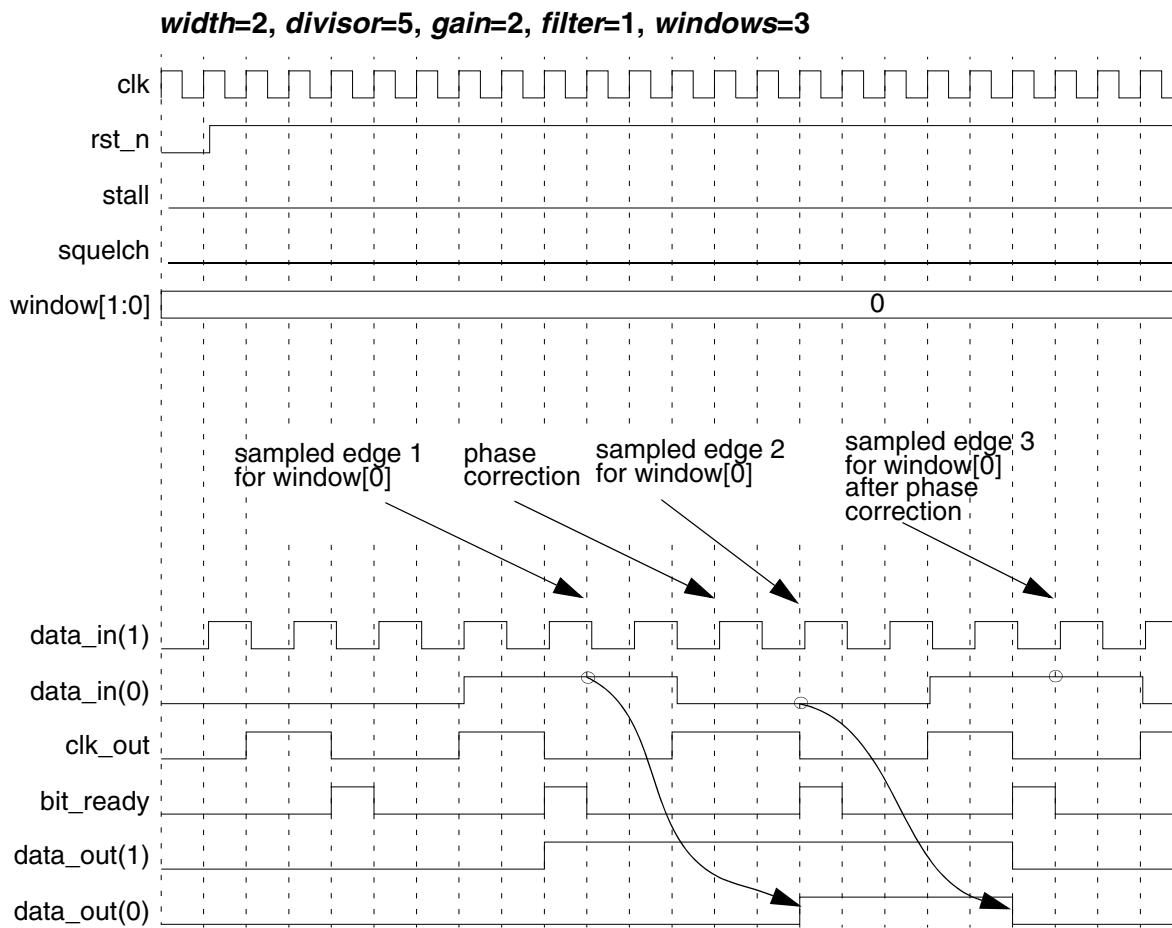


Figure 1-3 Window Shifting Sequence Concept

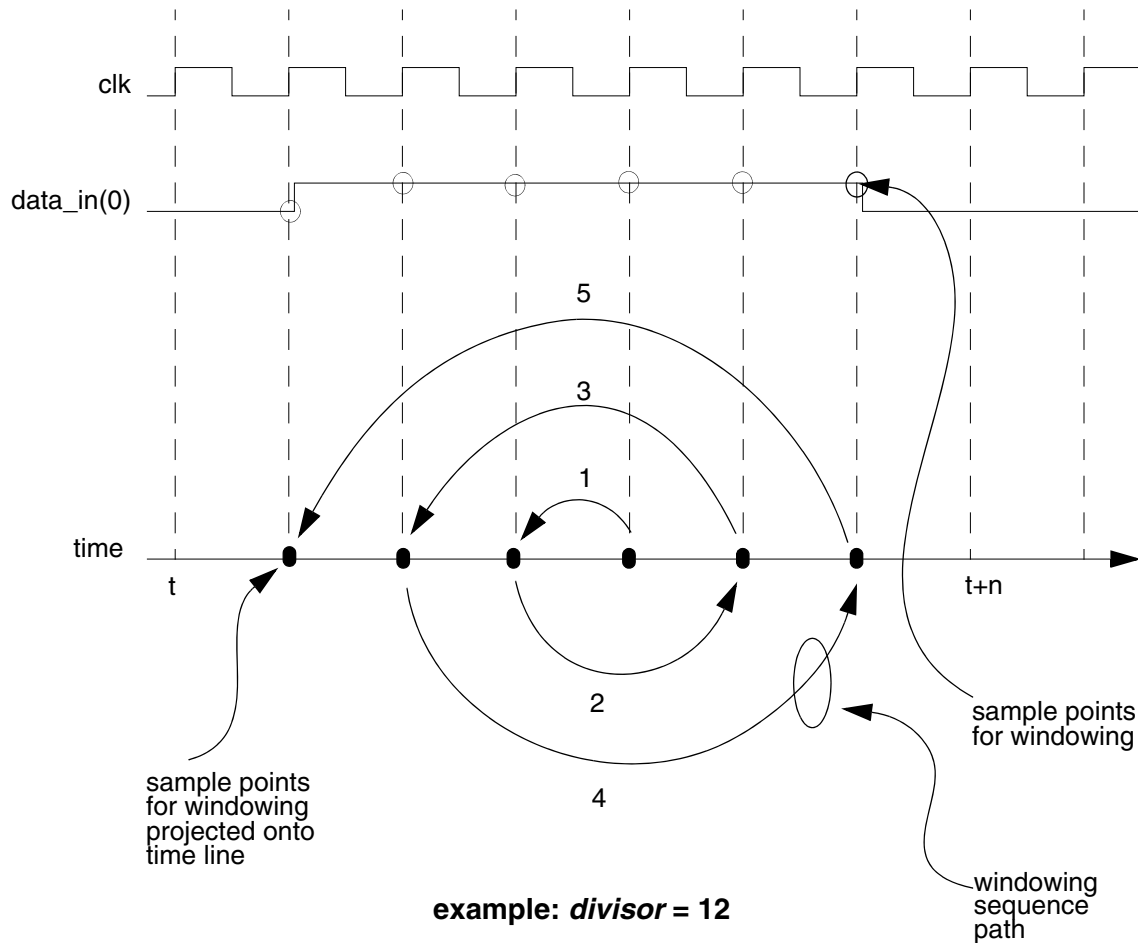


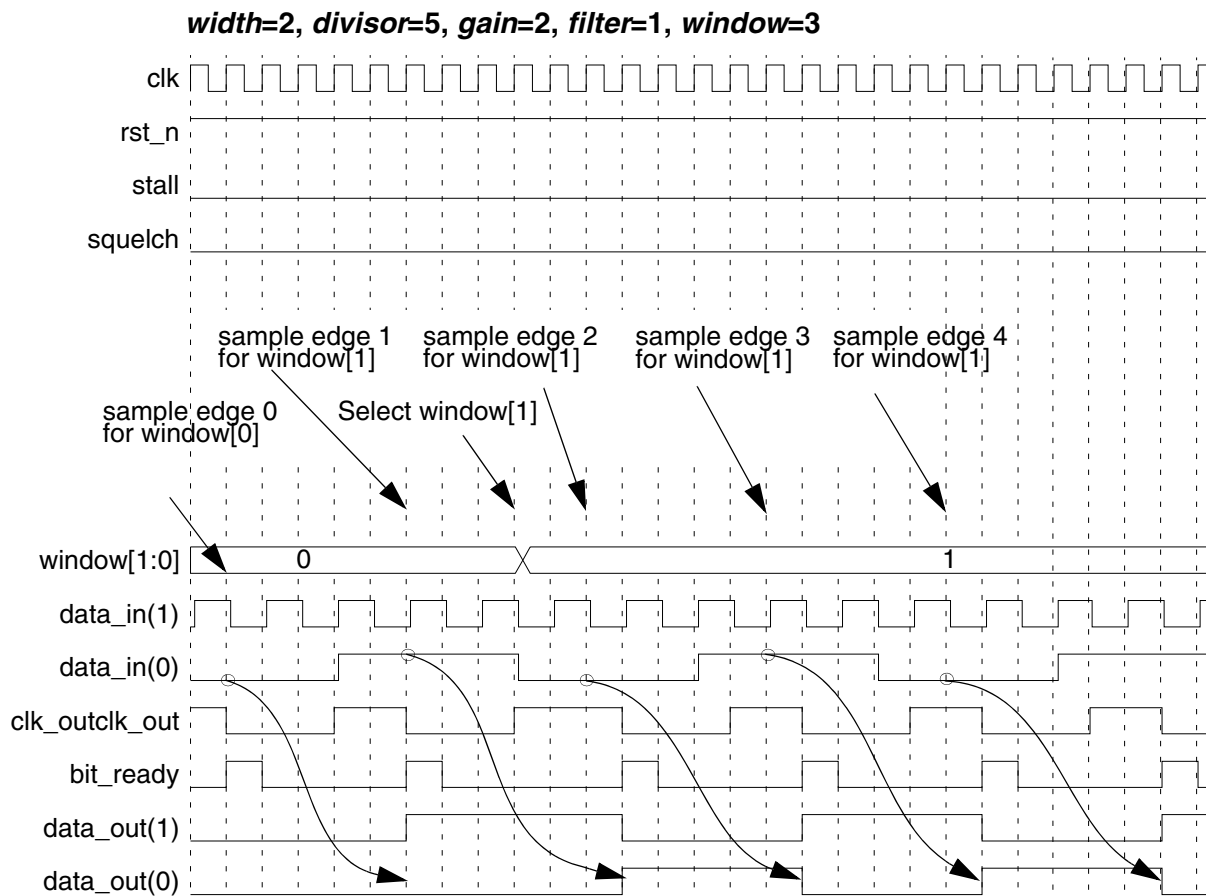
Figure 1-4 Functional Operation: Window shifting

Figure 1-5 Functional Operation: Squelch Active

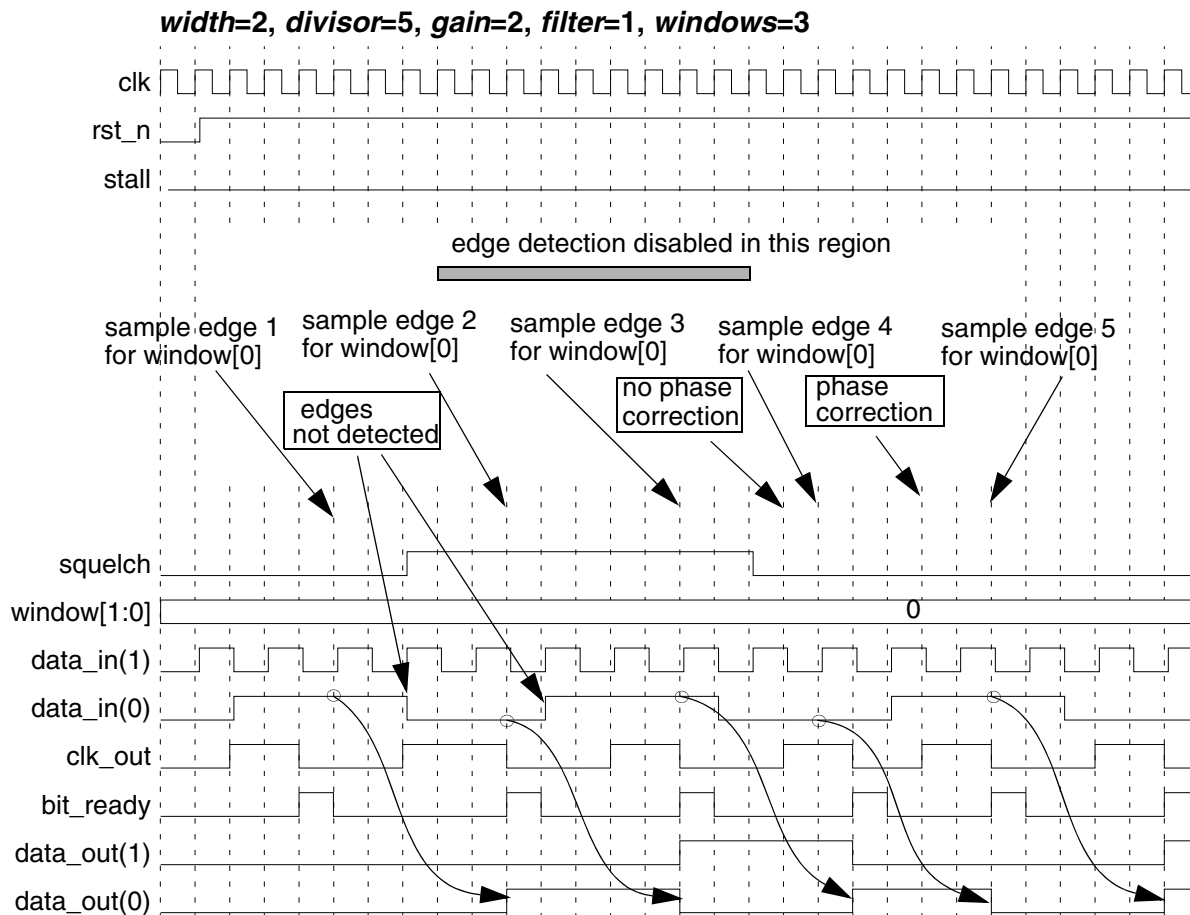
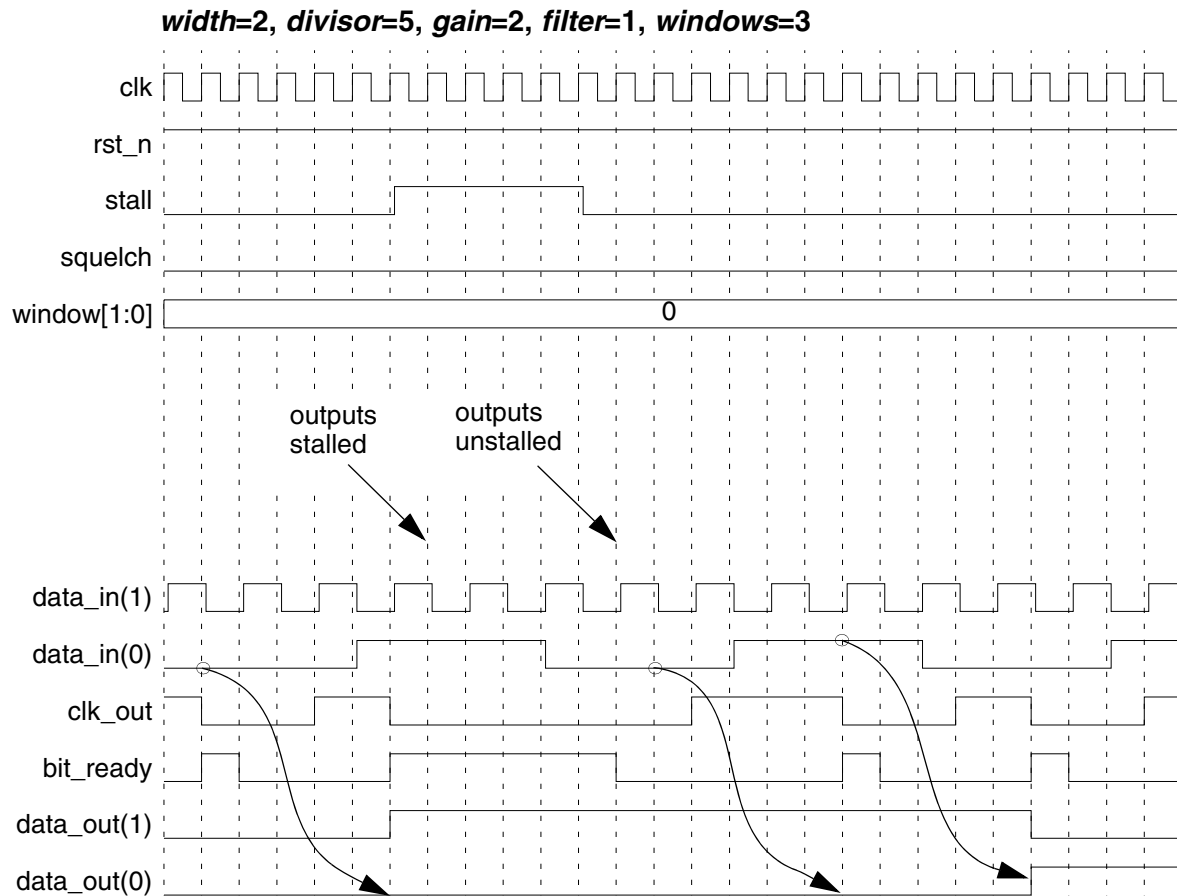


Figure 1-6 Functional Operation: Stall Active

Related Topics

- [Logic – Sequential Overview](#)
- [DesignWare Building Block IP User Guide](#)

HDL Usage Through Component Instantiation - VHDL

```
library IEEE,DWARE;
use IEEE.std_logic_1164.all;
use DWARE.DWpackages.all;
use DWARE.DW_foundation_comp.all;

entity DW_dp11_sd_inst is
  generic (inst_width : INTEGER := 1;
           inst_divisor : INTEGER := 5;
           inst_gain : INTEGER := 1;
           inst_filter : INTEGER := 2;
           inst_windows : INTEGER := 1);
  port (inst_clk : in std_logic;
        inst_rst_n : in std_logic;
        inst_stall : in std_logic;
        inst_squelch : in std_logic;
        inst_window: in std_logic_vector(bit_width(inst_windows)-1 downto 0);
        inst_data_in : in std_logic_vector(inst_width-1 downto 0);
        clk_out_inst : out std_logic;
        bit_ready_inst : out std_logic;
        data_out_inst : out std_logic_vector(inst_width-1 downto 0) );
end DW_dp11_sd_inst;

architecture inst of DW_dp11_sd_inst is
begin

  -- Instance of DW_dp11_sd
  U1 : DW_dp11_sd
    generic map ( width => inst_width, divisor => inst_divisor,
                  gain => inst_gain, filter => inst_filter,
                  windows => inst_windows )
    port map ( clk => inst_clk, rst_n => inst_rst_n, stall => inst_stall,
              squelch => inst_squelch, window => inst_window,
              data_in => inst_data_in, clk_out => clk_out_inst,
              bit_ready => bit_ready_inst, data_out => data_out_inst );
end inst;

-- pragma translate_off
configuration DW_dp11_sd_inst_cfg_inst of DW_dp11_sd_inst is
  for inst
  end for; -- inst
end DW_dp11_sd_inst_cfg_inst;
-- pragma translate_on
```

HDL Usage Through Component Instantiation - Verilog

```
module DW_dp11_sd_inst( inst_clk, inst_rst_n, inst_stall,
                        inst_squelch, inst_window, inst_data_in,
                        clk_out_inst, bit_ready_inst, data_out_inst );

    parameter inst_width = 1;
    parameter inst_divisor = 5;
    parameter inst_gain = 1;
    parameter inst_filter = 2;
    parameter inst_windows = 3;
    `define bit_width_windows 2 // ceil(log2(inst_windows))

    input inst_clk;
    input inst_rst_n;
    input inst_stall;
    input inst_squelch;
    input [`bit_width_windows-1 : 0] inst_window;
    input [inst_width-1 : 0] inst_data_in;
    output clk_out_inst;
    output bit_ready_inst;
    output [inst_width-1 : 0] data_out_inst;

    // Instance of DW_dp11_sd
    DW_dp11_sd #(inst_width, inst_divisor, inst_gain,
                 inst_filter, inst_windows)
        U1 ( .clk(inst_clk), .rst_n(inst_rst_n), .stall(inst_stall),
             .squelch(inst_squelch), .window(inst_window),
             .data_in(inst_data_in), .clk_out(clk_out_inst),
             .bit_ready(bit_ready_inst), .data_out(data_out_inst) );
endmodule
```

Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet with visible change bars, click [here](#).

Date	Release	Updates
September 2024	DWBB_202409.0	■ Updated date and version; no technical changes
December 2023	DWBB_202312.0	■ Updated date and version; no technical changes
July 2020	DWBB_201912.5	■ Adjusted content and title of “Suppressing Warning Messages During Verilog Simulation” on page 6 and added the DW_SUPPRESS_WARN macro
October 2019	DWBB_201903.5	■ Added the “Disabling Clock Monitor Messages” section
March 2019	DWBB_201903.0	■ Removed minPower designation from this datasheet
January 2019	DWBB_201806.5	■ Updated example in “HDL Usage Through Component Instantiation - VHDL” on page 12 ■ Added this Revision History table and the document links on this page

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

